

4주차 과제2

pe구조에 대한 이야기

패킹 분석 중에는 지금 보고 있는 주소가 파일의 어느 위치인지를 알아야 하고 디버거의 주소와 hexs 에디터의 오프셋이 왜 다른지도 알아야 하며 이 코드는 .text 섹션에 있는지 메모리에만 존재하고 파일에는 없는 영역인지, 덤프한 파일의 섹션 크기를 어떻게 수정해야 하는지 발견한 OEP를 PE 헤더의 어느 값에 기록해야 해야 하는지 등에 대한 정보를 전부 알아야 한다. 이러한 정보를 알기 위해서는 적어도 Raw Offset, RVA, VA, ImageBase, FileAlignment, SectionAlignment에 관한 정보를 알아야 한다. 따라서 이를 위해서는 PE구조에 대해서 어느 정도는 알고 있어야 한다.

PE구조는 디스크에서 대략 DOS Header, DOS Stub, PE Signature, COFF File Header, Optional Header, Section Table, 각 Section의 실제 데이터 순서로 저장되어있고 이를 조금 더 구체적으로 표현하면

+-----+

| IMAGE_DOS_HEADER |

| MZ |

| e_lfanew |

+-----+

| DOS Stub |

+-----+

| PE\WOW0 Signature |

+-----+

| IMAGE_FILE_HEADER |

+-----+

| IMAGE_OPTIONAL_HEADER |

| ImageBase |

| AddressOfEntryPoint |

| SectionAlignment |

| FileAlignment |

| SizeOfImage |

| DataDirectory |

+-----+

| IMAGE_SECTION_HEADER .text |

| IMAGE_SECTION_HEADER .rdata |

| IMAGE_SECTION_HEADER .data |

| ... |

+-----+

| .text Raw Data |

+-----+

| .rdata Raw Data |

+-----+

| .data Raw Data |

+-----+

pe헤더의 첫 주 바이트는 MZ 즉 0x 4D 5A이고 이 중에서 DOS Header의 오프셋 0x3C
에는 e_lfanew가 있다. 이 값은 PE 헤더가 시작되는 파일 오프셋으로 예를 들어 0x3C
위치에 다음 값 00 01 00 00이 있다고 하면 리틀엔디안임으로 이는 0x100으로 0x100에

가면 PE해더의 시작을 알리는 4바이트 50 45 00 00 즉 PEW0W0이 있다.

그 다음에는 optional header가 존재하는 데 이게 말이 선택이지 사실상 windows pe 실행 파일에서는 사실상 필수다.

optional header 구성 요소 중 imagebase는 실행 파일을 메모리에 배치하려는 기준 주소로 64비트 실행 파일에서는 흔히 ImageBase = 0x140000000와 같은 값을 사용하고 ALSR이 적용되면 이와 다른 값이 사용될 수도 있다.

AddressOfEntryPoint는 프로그램 실행을 시작할 코드의 RVA로 만약 ImageBase = 0x140000000 이고 AddressOfEntryPoint = 0x1000이며 ALSR을 무시한다면 실제 EntryPoint의 VA는 0x140001000이다. 여기서 중요한 것은 AddressOfEntryPoint가 RVA즉 상대적 주소 위치로 주어진다는 것이다.

그 다음 요소인 sectionalignment는 메모리에 섹션을 배치할 때 사용하는 정렬 단위로 일반적으로 4kb 즉 0x1000을 이용한다.

다음 요소인 FileAlignment은 디스크 파일에서 섹션 데이터를 정렬하는 단위로 일반적으로 0x200 즉 512바이트 경계에 맞추어 저장이 된다.

또한 이 때 Raw Offset, RVA, VA를 구분하는 게 가장 중요하다.

Raw Offset은 PE 파일을 hex스 에디터로 열었을 때의 파일 내부 위치다

RVA는 이미지가 메모리에 매핑된 뒤 ImageBase로부터 떨어진 거리다. 즉 imagebase로부터의 거리다.

VA는 실제 프로세스의 가상 메모리 주소다.

이 때 또 주의해야 할 점은 ASLR이 활성화되면 실제 ImageBase가 PE 헤더의 선호 ImageBase와 달라질 수 있으므로, 디버거에서는 현재 모듈의 실제 로드 베이스를 확인해야 한다는 것이다.

그 다음은 Raw offset을 계산하는 방법에 대해서 좀 더 자세히 알아보면 먼저 Name

= .text 섹션의 정보가 VirtualAddress= 0x1000, VirtualSize =0x1700, PointerToRawData = 0x400, SizeOfRawData= 0x1800라고 했을 때 찾으려는 RVA의 주소가 0x1234면 이는 $0x1000 < 0x1234 < 0x1000 + 0x1700$ 이기 때문에 이는 text영역에 있으며 이에 따라서 $\text{Raw offset} = \text{RVA} - \text{text section의 VirtualAddress} + \text{PointerToRawData} = 0x634$ 라는 걸 알 수 있다. 따라서 이 명령의 위치가 디스크에서 0x634에 있다는 사실을 알 수 있다.

반대로 Raw offset정보를 이용해 RVA를 계산시에는 만약 그 주소가 디스크의 특정 영역 내부에 존재하는 경우 $\text{RVA} = \text{Raw Offset} - \text{PointerToRawData} + \text{VirtualAddress}$ 와 같이 나타낼 수 있다.

그 다음에는 VirtualSize와 SizeOfRawData는 각 섹션의 Section Header, 즉 Section Table의 각 항목 안에 존재하는 값으로

SizeOfRawData는 디스크에 실제 저장된 크기로 FileAlignment에 맞춰 정렬된다.

VirtualSize는 메모리에 매핑되었을 때 섹션이 실제로 요구하는 크기다.

예를 들면 VirtualSize = 0x1735이고 SizeOfRawData = 0x1800일 수 있는 데 이전에 말했듯이 FileAlignment은 보통 512바이트 단위로 데이터를 저장하기 때문에 그렇다.

반대로 .bss처럼 초기값이 없는 전역 데이터가 많으면 다음과 같은 경우도 있다.

VirtualSize = 0x3000, SizeOfRawData = 0x200 이 경우에는 디스크에는 초기화 데이터 일부만 있고, 나머지 공간은 Windows 로더가 메모리에서 0으로 초기화한다.

패커는 이 차이를 적극적으로 이용할 수 있다. 디스크에서는 매우 작은 섹션이지만, 메모리에 올라간 뒤 큰 공간을 만들고 그곳에 원본 코드를 복원하는 방식이다.